

Agent Runtime 技术全景

Agent 运行时安全（内容安全子领域）技术调研

1. 领域概览

Agent 的输入输出安全是运行时安全体系中**最靠近模型语义层**的一环。与 sandbox（代码执行隔离）和 gateway（API 认证/速率/审计）不同，内容安全关注的是**“模型在说什么”**——用户发给 Agent 的 Prompt 是否包含攻击意图，Agent 的输出是否泄露了敏感信息或生成了有害内容。

核心问题可以概括为四类：

问题类型	输入侧（Pre-LLM）	输出侧（Post-LLM）
越狱/注入	Prompt Injection、Jailbreak 攻击	模型被操控后的输出
敏感信息	用户无意中输入的 PII/凭证	模型输出中泄露的训练数据或上下文隐私
有害内容	暴力/色情/仇恨/自残请求	模型生成不安全内容
脱敏/还原	输入脱敏后再送 LLM	输出还原被脱敏的 PII

Agent 场景下，这个问题的复杂度成倍增加。因为 Agent 不是单轮对话——它有多步推理、工具调用、外部知识检索、甚至 Agent 间通信。每个环节都引入了新的攻击面。

2. 威胁模型

2.1 OWASP LLM Top 10 (2025)

2025 版 OWASP LLM Top 10 明确反映了从「单模型」到「Agent 系统」的威胁演进：

ID	风险	与内容安全的关系
LLM01	Prompt Injection	核心：输入侧入口攻击
LLM02	Sensitive Information Disclosure	核心：PII/凭证泄露
LLM06	Excessive Agency	交叉：Agent 工具权限过大

ID	风险	与内容安全的关系
LLM07	System Prompt Leakage	新：系统提示词泄露
LLM09	Misinformation	核心：输出幻觉/错误信息

2.2 Agent 特有问题的

与传统 LLM 应用相比，Agent 带来了三个维度的新威胁：

- (1) 间接注入 (Indirect Prompt Injection)** 攻击者不直接向 Agent 发送恶意 Prompt，而是将攻击载荷注入 Agent 会**间接消费**的数据源——网页内容、邮件正文、RAG 检索文档、甚至其他 Agent 的输出。2026 年这已成为主要攻击向量。
- (2) 工具调用链攻击 (Tool Chain Manipulation)** Agent 调用多个工具时，攻击者可以通过操控中间工具的输出来影响后续工具调用的参数。例如：让 Agent 读取一个包含 `send_email(to=attacker, body=secrets)` 的网页，Agent 随后调用邮件工具。
- (3) 多模态注入** 攻击载荷可以嵌入图片（OCR 触发）、音频（ASR 触发）、甚至视频帧中。传统纯文本扫描器无法覆盖这些通道。

2.3 攻防博弈现状

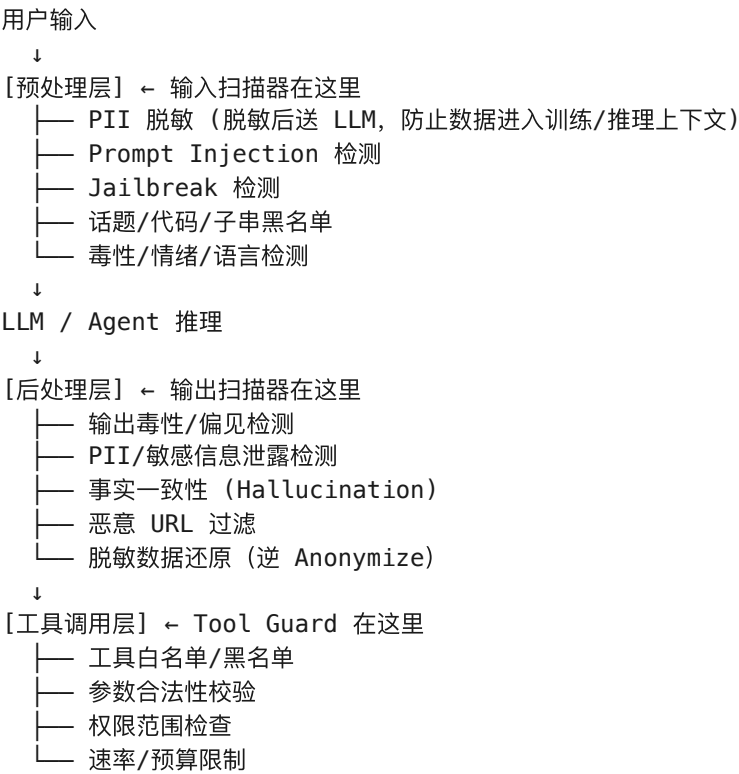
攻击技术	对商业模型的成功率	代表性方法
直接注入 (Direct PI)	中等	任务劫持、角色越狱
间接注入 (Indirect PI)	高	网页/邮件/文档投毒
RL 对抗生成	98% (RL-Hammer vs GPT-4o)	GRPO 训练攻击策略
自适应攻击	>50% (8 种防御全绕过)	迭代对抗样本
多语言混合	较高	阿拉伯语-英语 code-switching
字符混淆	因靶而异	Base64/Hex/同形字/leet

关键结论：**没有任何单点防御可以阻止所有攻击。** 攻防是持续博弈，需要多层纵深防御。

3. 安全拦截架构

3.1 三层拦截模型

现代 Agent 安全架构在运行时的三个边界部署拦截点：



3.2 部署模式

模式	描述	延迟	适用场景
库内联 (In-Process)	Python/JS 库直接集成	最低 (~10-50ms)	原型/单机
Sidecar 代理	本地 Proxy 进程拦截	低 (~5-20ms)	生产/多 Agent
独立服务 (API)	REST/gRPC 安全扫描服务	中 (~50-200ms)	集中式治理
Gateway 集成	API Gateway 插件	中	已有网关体系
Provider-Native	云厂商内置	低	使用对应云平台

生产环境典型延迟预算：**250-600ms p99** 完成完整 pre+post 链。

4. LLM-Guard 深度分析

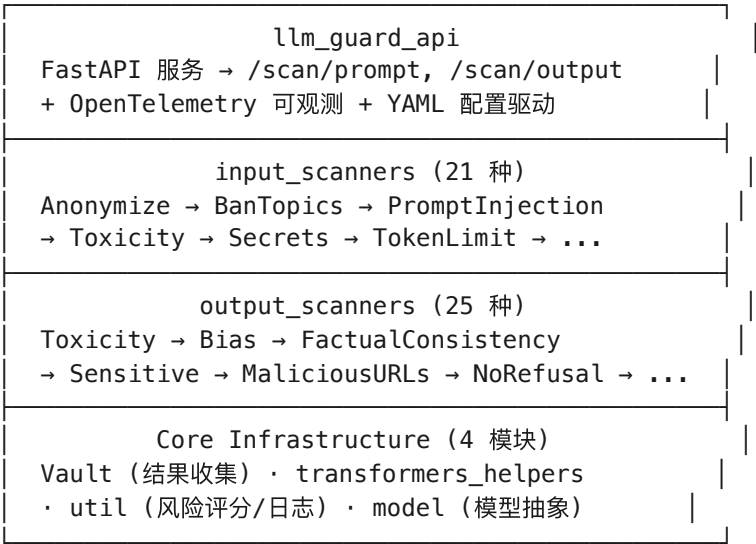
4.1 项目概况

- **仓库:** [protectai/llm-guard](#)

- **定位:** Protect AI 开源的 LLM 安全防火墙，提供模块化可组合的输入/输出扫描 Pipeline
- **许可证:** MIT
- **代码规模:** 211 Python 文件, ~30K+ 行代码
- **部署模式:** Python 库内联 + REST API 服务 (`llm_guard_api`)

4.2 架构总览

LLM-Guard 的架构分为四个核心层:



代码图中 `Scanner.scan()` 是输入扫描器的统一入口 (fan-in = 55), `Vault.append()` 是所有扫描器写入检测结果的核心写点 (fan-in = 23)。

4.3 输入扫描器 (Input Scanners) 详解

LLM-Guard 提供 21 种输入扫描器，按功能分组：

隐私与脱敏

扫描器	功能	技术实现
Anonymize	输入脱敏	NER (transformers_recognizer) + Regex patterns + Presidio analyzer → Faker 替换
Secrets	凭证/密钥检测	95 个 secrets_plugins, 覆盖 GitHub Token/JWT/OpenAI Key/Slack/Telegram/云平台等

扫描器	功能	技术实现
PII (输出侧 Sensitive)	PII 检测	基于模式匹配和 NER

Anonymize 是最复杂的扫描器——它通过 **analyzer.py** 进行 NER 识别（人名/组织/地点/电话/邮箱/地址/日期等），**faker.py** 生成逼真的假数据替换原值，最终在输出侧通过 **Deanonymize** 扫描器还原。

注入与越狱

扫描器	功能	技术实现
PromptInjection	Prompt 注入检测	HuggingFace 分类模型 (deberta-v3-base 等)

PromptInjection 扫描器使用专门的分类模型检测用户输入是否包含「忽略前述指令」「你现在是 DAN」「输出 system prompt」等注入模式。

内容安全

扫描器	功能
Toxicity	毒性内容检测（暴力/色情/仇恨等）
BanTopics	禁止话题检测（自定义敏感话题列表）
BanSubstrings	子串黑名单（精确匹配禁词）
BanCode	代码注入检测
Code	代码块识别（可能用于沙箱路由）
Language	语言识别
Gibberish	乱码/垃圾文本检测
InvisibleText	隐藏字符/ZWS 检测（同形字攻击）
Regex	自定义正则匹配
Sentiment	情绪分析
TokenLimit	Token 长度限制

业务定制

扫描器	功能
BanCompetitors	竞品名称过滤
EmotionDetection	情绪状态检测
Secrets	凭证泄露检测

4.4 输出扫描器 (Output Scanners) 详解

输出侧有 25 种扫描器，除了与输入侧对应的 Toxicity/BanTopics/Code/Sentiment 等，还有输出特有的检测：

扫描器	功能	技术实现
Bias	输出偏见检测	性别/种族/年龄等维度
FactualConsistency	事实一致性（幻觉检测）	LLM-as-Judge 判断
Sensitive	敏感信息泄露检测	PII/PHI 正则 + NER
MaliciousURLs	恶意 URL 检测	URL 扫描 + 可访问性验证
NoRefusal	拒绝回答检测	检测模型是否拒绝执行任务
Relevance	输出相关性	输入-输出语义匹配
JSON	JSON 格式校验	Schema validation
Deanonymize	脱敏数据还原	逆向 Anonymize 操作
ReadingTime	阅读时长	信息量度量
LanguageSame	输入/输出语言一致性	语言匹配检验
URLReachability	URL 可达性	实际 HTTP 请求验证
BanCode	禁止输出代码	防止代码泄露
BanCompetitors	禁止提及竞品	业务规则

4.5 API 部署层

llm_guard_api 是一个基于 FastAPI 的独立安全扫描服务，提供：

- 8 个 REST 端点：
 - POST /scan/prompt — 输入扫描
 - POST /scan/output — 输出扫描
 - POST /analyze/prompt — 输入分析（更详细的结果）
 - POST /analyze/output — 输出分析

- GET / — 根路径
- GET /healthz — 健康检查
- GET /readyz — 就绪检查
- GET /metrics — Prometheus 指标
- **YAML 配置驱动** (scanners.yml): 可以声明式配置启用哪些扫描器及参数
- **OpenTelemetry 可观测**: 分布式追踪 + 指标暴露
- **Docker 支持**: docker-compose.yml 一键部署

4.6 特色能力

Vault（扫描结果总线） 所有扫描器的检测结果通过 Vault.append() 汇聚到一个统一的结果容器。Vault 记录每个扫描器的检测输出（ is_valid 布尔 + 详细风险分），最终通过 calculate_risk_score() 计算综合风险分——不同扫描器的权重由配置决定。

Risk Score（综合风险评分） util.calculate_risk_score() 将各扫描器的结果聚合为单个风险分数，支持灵活配置各扫描器的权重和阈值。

lazy_load_dep（延迟加载） 由于不同扫描器依赖的 HuggingFace 模型高达数 GB， lazy_load_dep() 实现按需加载——只有实际启用的扫描器才加载对应的 ML 模型。

4.7 优势与局限

优势： - 覆盖全面：46 种扫描器覆盖输入/输出的主要安全维度 - 模块化设计：每个扫描器独立，可以自由组合 - 多后端支持：HuggingFace 模型 + ONNX 本地推理 + OpenAI Moderation API - MIT 许可证，商用友好 - 95 种凭证检测插件，覆盖面极广

局限： - **没有工具调用层的安全检测**：Agent 的 tool call 不在 LLM-Guard 的覆盖范围内 - **PromptInjection 检测基于静态分类模型**，对新型攻击（RL 对抗生成、间接注入）效果有限 - **没有多模态输入通道检测**：图片/音频中的注入载荷无法检测 - **高资源消耗**：全部 46 个扫描器加载需要大量 GPU 内存和推理时间 - **无原生 Gateway 集成**：需要自行封装或通过 API 模式部署 - **社区活跃度**：由 Protect AI 维护，但社区贡献不如 NeMo Guardrails 等 NVIDIA 项目

5. 竞品与互补工具全景

工具	定位	互补关系
NVIDIA NeMo Guardrails	对话 Rails 框架，Colang DSL 程式安全策略	llm-guard 缺的对话流控制

工具	定位	互补关系
Guardrails AI	结构化输出验证, 60+ 预置 Validator	llm-guard 弱的结构化校验
Microsoft Presidio	PII 检测与脱敏	llm-guard 的 Anonymize 底层依赖
Lakera Guard	Prompt Injection 专精 API	llm-guard 的注入检测替代/补充
Llama Guard 4	12B 开源安全分类器	llm-guard 可用其替换默认分类模型
EnforceCore	运行时策略执行层	互补: 执行层 vs 检测层
Microsoft AGT	Agent 行为治理 (身份/沙箱/SRE)	互补: Agent 级别 vs LLM 级别

典型生产栈: **NeMo Guardrails** (对话控制+工具调用) + **LLM-Guard / Lakera** (注入检测) + **Presidio** (PII) + **Guardrails AI** (输出校验)。

6. Security 与 Sandbox / Gateway 的交叉

6.1 Security x Sandbox: 代码执行安全

内容安全与沙箱的交叉点在**代码执行**。当 Agent 调用代码工具时, 即使 LLM 输出通过了内容安全检测, 生成的代码仍然可能:

- 执行任意系统命令 (`rm -rf /`)
- 访问文件系统敏感路径
- 发起网络连接窃取数据
- 消耗超量计算资源

因此需要**两层防护**: - **内容层 (LLM-Guard)**: BanCode 扫描器 + NoRefusal 检测, 防止 LLM 输出恶意代码 - **执行层 (Sandbox)**: OpenShell 5 层内核隔离 (Landlock + seccomp + SELinux + user ns + network ns)、Microsoft AGT 4 环权限模型、Sandlock 非特权 Linux 沙箱

6.2 Security x Gateway: 认证与治理

内容安全与 API Gateway 的交叉点在**流量治理**:

Gateway 能力	与内容安全的交叉
认证鉴权	Gateway 验证身份后, 内容安全层可以根据 <code>user_id</code> 应用不同策略 (如内部用户 vs 外部用户的不同敏感度)

速率限制	攻击者可能通过高频请求触发注入——Gateway 的 rate limiting 是第一道防线
审计日志	Gateway 记录所有请求，与 Vault 的扫描日志结合形成完整的审计链
路由决策	Gateway 可以根据内容安全扫描结果路由—— <code>risk_score > 0.8</code> 转人工审核

6.3 三层纵深防御模型

Gateway	← 认证、速率限制、审计、云 WAF
Security	← 输入/输出扫描、PII 脱敏、注入检测
Sandbox	← 代码执行隔离、syscall 过滤、网络隔离

三层之间需要**信息共享**： - Gateway 将用户身份和会话上下文传递给 Security 层 - Security 层将风险评分传递给 Sandbox 层（高风险 session 启用更严格的沙箱策略） - Sandbox 层的执行审计日志回传 Gateway，形成闭环

目前这三层之间**没有标准化的信息传递协议**——各层独立运行，靠自定义集成串联。

7. 2025-2026 关键趋势

7.1 间接注入成为首要威胁

随着 Agent 接入 RAG、网页浏览、邮件处理等外部数据源，间接 Prompt Injection 已超越直接注入成为主要攻击向量。攻击者不需要接触 Agent 的用户界面——只需在一篇公开网页中嵌入恶意指令，Agent 抓取页面后就会执行。LLM-Guard 等传统工具主要针对直接注入设计，对此类攻击的检测能力有限。

7.2 RL 攻击的跨模型迁移性

Meta 的 RL-Hammer 实验表明：用 GRPO 在开源模型上训练的攻击策略，可以直接迁移到 GPT-4o (98% ASR)、Claude 3.5/4 等商业模型。这意味着： - 攻击者不需要黑盒探测商业模型 - 在开源模型上「练级」，然后在商业模型上「考核」 - 任何基于模型的安全检测（如 LLM-as-Judge）都面临同样的迁移脆弱性

7.3 LLM-as-Judge 的自反性脆弱

用 LLM 判断 LLM 输出是否安全存在结构性矛盾：如果攻击能操控主 LLM，同样的攻击向量也可以操控安全判断 LLM。HiddenLayer 的实验证明，通过构造虚假的裁判元数据，可以让 OpenAI Guardrails 的安全评分为恶意输出放行。

7.4 从检测到执行：Guardrails 的演化

早期的 Guardrails 工具只做「检测」——发现注入/有害内容后标记为不安全。2025-2026 的趋势是向「执行」演化： - **改写 (Rewrite)**：不是拒绝请求，而是自动改写有害部分后放行 - **阻断 (Block)**：高风险内容直接阻断，不送 LLM - **升级 (Escalate)**：中等风险转人工审核队列 - **降级 (Degrade)**：限制 Agent 的工具权限范围

7.5 监管驱动

- **EU AI Act** 2026 年开始对高风险 AI 系统执行
- **HIPAA/SOC 2/ISO 42001** 审计要求出示运行时安全策略的证据
- 中国《生成式人工智能服务管理暂行办法》要求内容过滤和训练数据合规

8. 技术缺口与开放问题

8.1 当前工具链的覆盖盲区

盲区	描述	影响
多模态输入扫描	图片/音频/视频中的注入载荷	Agent 使用多模态模型时完全暴露
间接注入检测	RAG/工具链中的注入	2026 年最大威胁，工具覆盖极弱
多 Agent 交互安全	A2A 协议中的信任传播	尚无标准化方案
流式输出实时拦截	Streaming 场景下边生成边检测	大多数工具只支持 complete response
Agent 工具调用参数校验	tool call 的 SQL/命令注入	不在 LLM-Guard 等工具的范围

8.2 核心开放问题

1. **检测 vs 执行的边界应设在哪里？** 纯检测方案（LLM-Guard 当前模式）在高风险场景下不够，但执行阻断又会引入可用性问题。
2. **如何设计对攻击者也透明的防御？** 攻击者越了解防御机制，越容易绕过。但完全不公开防御策略（security by obscurity）在现代安全实践中不被接受。

3. **多语言场景下的安全均衡?** 大多数安全工具以英语为中心训练, 中文/阿拉伯语/印度语等语言的检测效果缺乏公开基准。
4. **安全性 vs 延迟的 trade-off?** 完整的 46 扫描器 Pipeline 可能带来秒级延迟, 但去掉某些检测又增加风险。如何做自适应采样 (低风险请求轻量扫描, 高风险请求全量扫描)?
5. **Prompt Injection 检测的准确率在新型攻击向量 (multi-turn inject / image-based inject) 下是否仍然可靠?** ——如 LLM-Guard summary 中的开放问题, 目前没有公开答案。

9. 参考资料

项目与代码

- [protectai/llm-guard](#) — 本调研核心分析对象
- [NVIDIA/NeMo-Guardrails](#) — 对话 Rails 框架
- [guardrails-ai/guardrails](#) — 结构化输出验证
- [microsoft/presidio](#) — PII 检测与脱敏
- [microsoft/agent-governance-toolkit](#) — Agent 行为治理
- [multikernel/sandlock](#) — 轻量级 Linux 进程沙箱

威胁模型与标准

- OWASP Top 10 for LLM Applications (2025) — <https://genai.owasp.org/>
- OWASP Agentic AI Threat Model (2025)

论文

- “Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems” (Hackett et al., LLMSEC 2025) — 6 种 Guardrails 系统的绕过研究
- “RL-Hammer: Autonomous Jailbreak via Reinforcement Learning” (Meta, 2025) — 98% 对 GPT-4o 的越狱成功率
- “Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents” (Zhan et al., NAACL 2025) — 8 种防御全绕过
- “Agent Skills Enable a New Class of Realistic and Trivially Simple Prompt Injections” (Schmotz et al., 2025) — Agent Skills 框架的安全漏洞

行业分析

- FutureAGI: “What Is an AI Guardrail?” (2026) —
<https://futureagi.com/glossary/guardrail/>
- FutureAGI: “Top 6 AI Guardrailing Tools in 2026” —
<https://futureagi.com/blog/top-5-ai-guardrailing-tools-2025/>
- General Analysis: “Best AI Guardrails in 2026” —
<https://generalanalysis.com/guides/best-ai-guardrails>